TECMILENIO

FASE II

Documento de validación y pruebas del sistema inteligente de priorización de eventos de ciberseguridad

Materia:	*Gestión de Proyectos de Inteligencia Artificial*
Actividad:	*Fase II - Documento de validación y pruebas*
Alumno(a):	*Alejandro Bárcenas Godínez*
Matrícula:	*7182682*
Docente:	*Luis Ariel Vázquez Piña*
Fecha de entrega:	*[08 / 11 / 2026]*

Declaración de Uso de Inteligencia Artificial

El alumno debe indicar de manera honesta si utilizó herramientas de Inteligencia Artificial (IA) para el desarrollo de esta actividad. Completa la siguiente tabla:

Herramienta de IA utilizada	¿La usaste?	Propósito / Descripción de uso
ChatGPT / OpenAI	☒Sí ☐ No	Apoyo en la estructuración y revisión del informe, documentación de Python, interpretación crítica de métricas y organización de evidencias de FastAPI y Docker. Los resultados técnicos fueron ejecutados y verificados por mí mismo.
Claude (Anthropic)	☐Sí ☒ No	
Gemini (Google)	☐Sí ☒ No	
Copilot (Microsoft)	☐Sí ☒ No	
Otra (especifica): Miro	☐Sí ☒ No	

Desarrollo de Contenido

1. Introducción

El presente documento reúne la validación experimental y las pruebas funcionales del prototipo académico de priorización de eventos de ciberseguridad desarrollado en la Fase II. La solución integra un modelo de clasificación, una API FastAPI, una interfaz web y una imagen Docker ejecutable localmente. El objetivo de esta evaluación es demostrar el funcionamiento de sus componentes, documentar resultados observados y delimitar sus alcances.

El modelo utiliza etiquetas de eventos derivadas de CSE-CIC-IDS2018 y tres variables de contexto sintético: criticidad del activo, exposición a Internet y participación de una cuenta privilegiada. Las prioridades Baja, Media, Alta y Crítica se asignaron originalmente mediante una política académica de puntuación. Por ello, las pruebas miden principalmente la capacidad del modelo para reproducir dicha política y la consistencia de la integración técnica; no acreditan capacidad de detección autónoma de amenazas reales.

2. Desarrollo

2.1 Objetivo y alcance de la validación

Verificar que el pipeline exportado desde Google Colab puede cargarse en Windows, producir predicciones consistentes, responder a solicitudes HTTP mediante FastAPI y funcionar dentro de un contenedor Docker. Se comprueba además el rechazo de entradas inválidas. Las pruebas son locales y académicas; no se efectuó un despliegue real en la nube ni una evaluación operativa en un centro de seguridad.

2.2 Preparación del conjunto y modelo

En Google Colab se seleccionaron inicialmente 18,000 registros de siete categorías de tráfico. Tras la limpieza y eliminación de duplicados quedaron 13,523 registros. Se añadieron variables contextuales sintéticas y se asignaron prioridades con reglas explícitas. La partición estratificada reservó 9,465 registros para entrenamiento, 2,029 para validación y 2,029 para prueba.

Prioridad	Registros	Porcentaje (%)
Baja	3070	22.70
Media	6589	48.72
Alta	2720	20.11
Crítica	1144	8.46

Figura 1. Distribución general de prioridades. Fuente: Elaboración propia.

Se compararon regresión logística, árbol de decisión y Random Forest. Los tres alcanzaron métricas de 1.0000 en validación. Se seleccionó regresión logística para la exportación. Dado que las etiquetas objetivo proceden de reglas basadas en las mismas variables disponibles para el modelo, este resultado era plausible y debe interpretarse como reproducción de la política, no como evidencia de generalización frente a ataques desconocidos.

2.3 Resultados sobre el conjunto de prueba

La matriz de confusión presentada en la ejecución del notebook mostró 461 registros Baja, 988 Media, 408 Alta y 172 Crítica sobre la diagonal, sin errores en ese conjunto. Estos valores corresponden a la evaluación académica reservada; no deben extrapolarse a tráfico real de producción.

Indicador	Resultado observado
Registros de prueba	2029
Accuracy	1.00
Precision macro	1.00
Recall macro	1.00
F1-score macro	1.00
Eventos Críticos evaluados	172
Eventos Críticos mal clasificados	0
Casos extremos del notebook	6 de 6 correctos

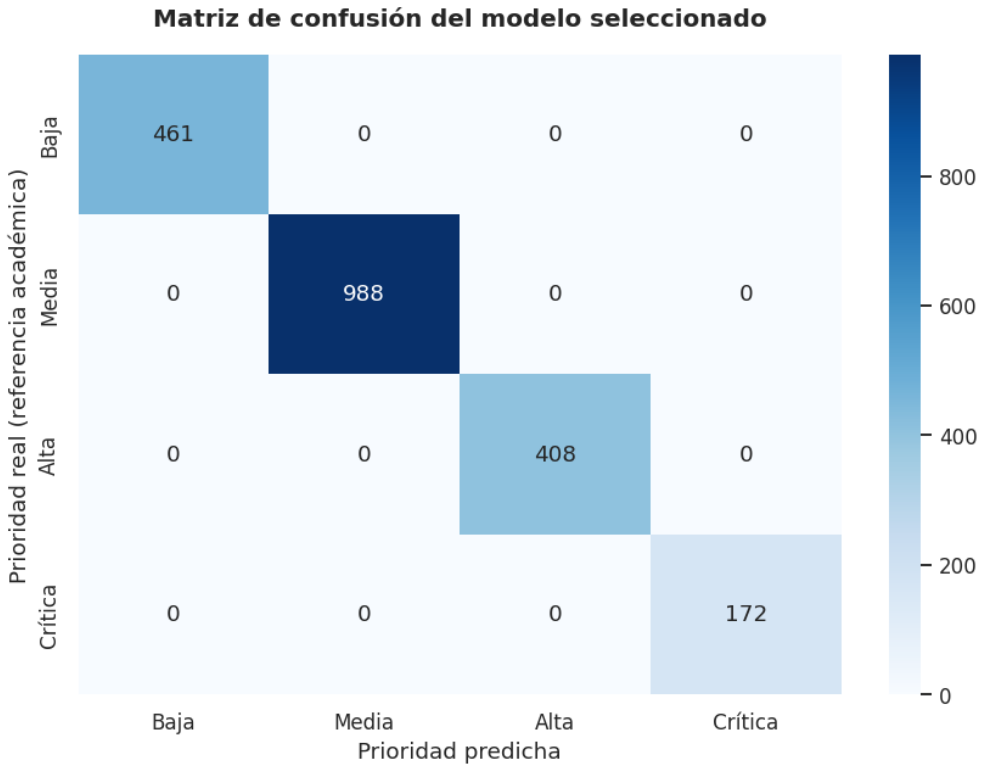


Figura 2. Matriz de confusión del modelo seleccionado sobre 2,029 registros de prueba. Fuente: Elaboración propia.

2.4 Integración de la API y ejecución en Docker

El backend expone GET /health para comprobar disponibilidad y POST /predict para recibir las cuatro variables de entrada. La interfaz HTML, CSS y JavaScript envía solicitudes JSON a la API y muestra la prioridad sugerida con una advertencia de supervisión humana. El modelo se carga desde modelo/modelo_priorizacion.joblib mediante Joblib.

La imagen fase-ii-ciberseguridad:1.0 se construyó satisfactoriamente a partir de python:3.13-slim. El contenedor fase-ii-app se ejecutó publicando el puerto local 8001 hacia el puerto 8000 interno. La comprobación posterior con docker ps mostró el contenedor activo durante 19 horas; GET /health respondió con estado «operativo». La imagen registrada presentó un uso en disco de aproximadamente 668 MB en ese entorno.

2.5 Pruebas funcionales y casos extremos

Caso	Entrada o condición	Resultado observado
P01 – Baja	Benign; activo Baja; Internet 0; privilegiada 0	HTTP 200; Baja
P02 – Media	Benign; activo Alta; Internet 0; privilegiada 0	HTTP 200; Media
P03 – Alta	Benign; activo Alta; Internet 1; privilegiada 1	HTTP 200; Alta
P04 – Crítica	SSH-Bruteforce; activo Alta; Internet 1; privilegiada 1	HTTP 200; Crítica
P05 – Fuera de rango	Exposicion_Internet = 5	HTTP 422; solicitud rechazada
P06 – Categoría desconocida	Label = Ataque-Desconocido	HTTP 422; solicitud rechazada

Las seis pruebas anteriores fueron ejecutadas desde la documentación interactiva de FastAPI y/o desde el formulario web, con evidencia visual proporcionada durante el desarrollo. Los errores HTTP 422 son resultados esperados: confirman que el esquema de entrada impide procesar valores fuera del dominio admitido.

Validación complementaria de casos extremos en Google Colab

Adicionalmente, durante la etapa de evaluación experimental del modelo en Google Colab, se realizaron seis pruebas de casos extremos mediante combinaciones específicas del tipo de evento, la criticidad del activo, la exposición a Internet y la participación de cuentas privilegiadas. Estas pruebas tuvieron como propósito comprobar la consistencia de las predicciones ante escenarios con diferentes condiciones de riesgo, incluyendo eventos benignos y ataques de fuerza bruta o denegación de servicio.

Los resultados mostraron una coincidencia de 6 de 6 casos (100 %) entre las prioridades esperadas conforme a la política académica y las prioridades predichas por el modelo seleccionado. Este resultado complementa las pruebas funcionales ejecutadas posteriormente mediante FastAPI y Docker, al verificar el comportamiento del clasificador ante combinaciones previamente definidas.

Es importante distinguir ambos procedimientos: las seis pruebas experimentales de Google Colab evaluaron la correspondencia entre el modelo y la política de priorización, mientras que las seis pruebas realizadas mediante la API comprobaron cuatro respuestas válidas (HTTP 200) y dos mecanismos de rechazo de entradas incorrectas (HTTP 422). Por tanto, ambos conjuntos de pruebas son complementarios y no deben interpretarse como una validación exhaustiva del sistema frente a incidentes reales de ciberseguridad.

2.6 Reproducibilidad y trazabilidad

El repositorio público https://github.com/Aletz44/Fase-II-Ciberseguridad-ABG contiene el código del backend, frontend, Dockerfile, requirements.txt, archivos de configuración, modelo exportado, README.md y notebook Fase_II_ABG.ipynb. La rama main quedó sincronizada tras el commit 834ac83. Se verificó la construcción y ejecución desde el directorio local; no consta todavía una reconstrucción independiente desde una clonación limpia.

2.7 Limitaciones y consideraciones críticas

La variable Label del conjunto de datos identifica el tipo de evento y se utiliza directamente como predictor, junto con los mismos atributos sintéticos empleados para definir la prioridad. Esto genera una relación estrecha entre predictores y objetivo. Las métricas perfectas, por tanto, no permiten afirmar que el modelo descubra patrones de riesgo no contenidos en la política original ni que supere dicha política.

La limpieza redujo considerablemente las clases FTP-BruteForce y DoS attacks-SlowHTTPTest, que conservaron 42 registros cada una, limitando su representatividad. La muestra seleccionada no equivale a la distribución completa de CSE-CIC-IDS2018. No se evaluaron ataques inéditos, deriva temporal, calibración probabilística, seguridad del servicio expuesto públicamente, ni calidad de decisiones humanas reales.

El mensaje de supervisión humana aparece en la interfaz, pero el prototipo no implementa almacenamiento ni auditoría de decisiones de analistas. Las pruebas HTTP 422 verifican validación básica de esquema, no una auditoría integral de seguridad. La ejecución en Docker fue local; el manual de despliegue en la nube corresponde a un entregable separado.

3. Conclusiones

La Fase II demostró una cadena funcional de extremo a extremo: formulario web, API FastAPI, pipeline de clasificación y respuesta visual, todo ejecutado localmente dentro de Docker. Se comprobaron las cuatro prioridades esperadas y dos escenarios de rechazo de entradas inválidas. El repositorio GitHub permite localizar los componentes principales del prototipo y su notebook de entrenamiento.

El principal resultado técnico es la portabilidad e integración reproducible en el entorno probado. El desempeño perfecto del clasificador es coherente con el origen reglado de las prioridades y no constituye evidencia de precisión perfecta en ciberseguridad real. La solución debe considerarse un prototipo de apoyo académico sujeto a revisión humana y a validaciones adicionales antes de cualquier utilización operativa.

Anexo A

Las siguientes figuras corresponden a las gráficas del notebook y a capturas reales de las pruebas realizadas en el entorno local y Docker. Se presentan como evidencia visual de los resultados descritos.

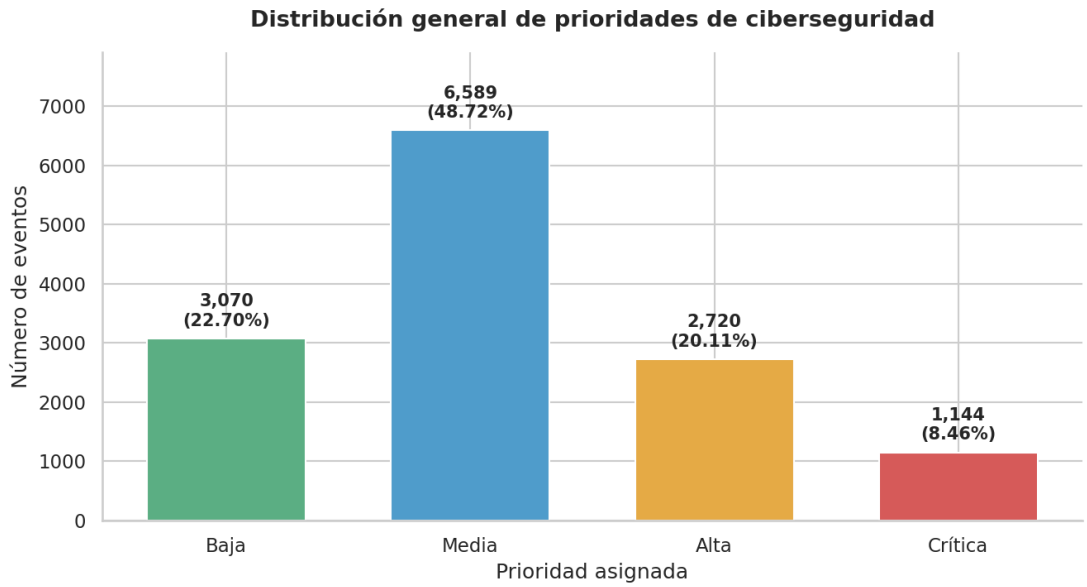


Figura 3. Distribución de prioridades asignadas en el conjunto académico. Fuente: ejecución del notebook en Google Colab.

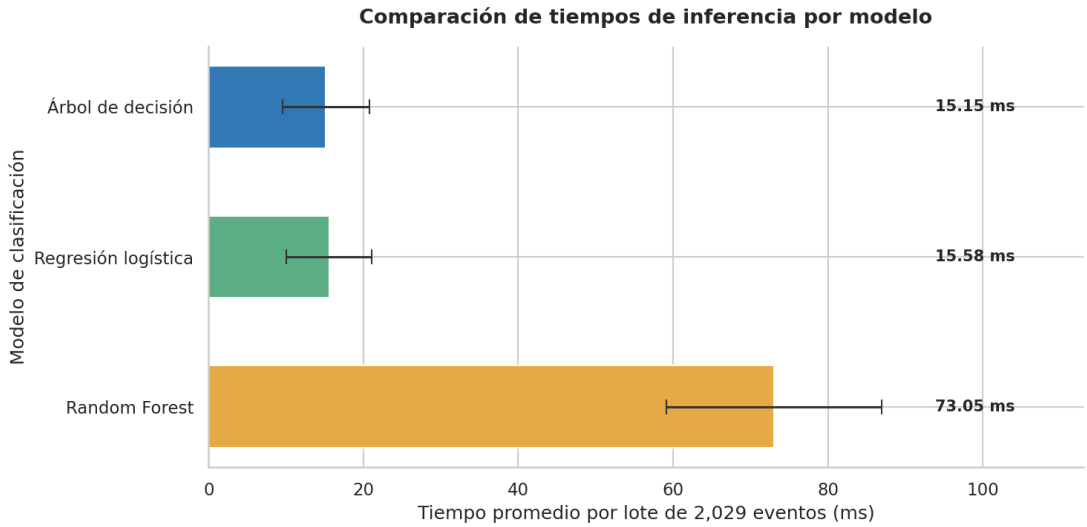


Figura 4. Comparación de tiempos de inferencia de los modelos evaluados. Fuente: ejecución del notebook.

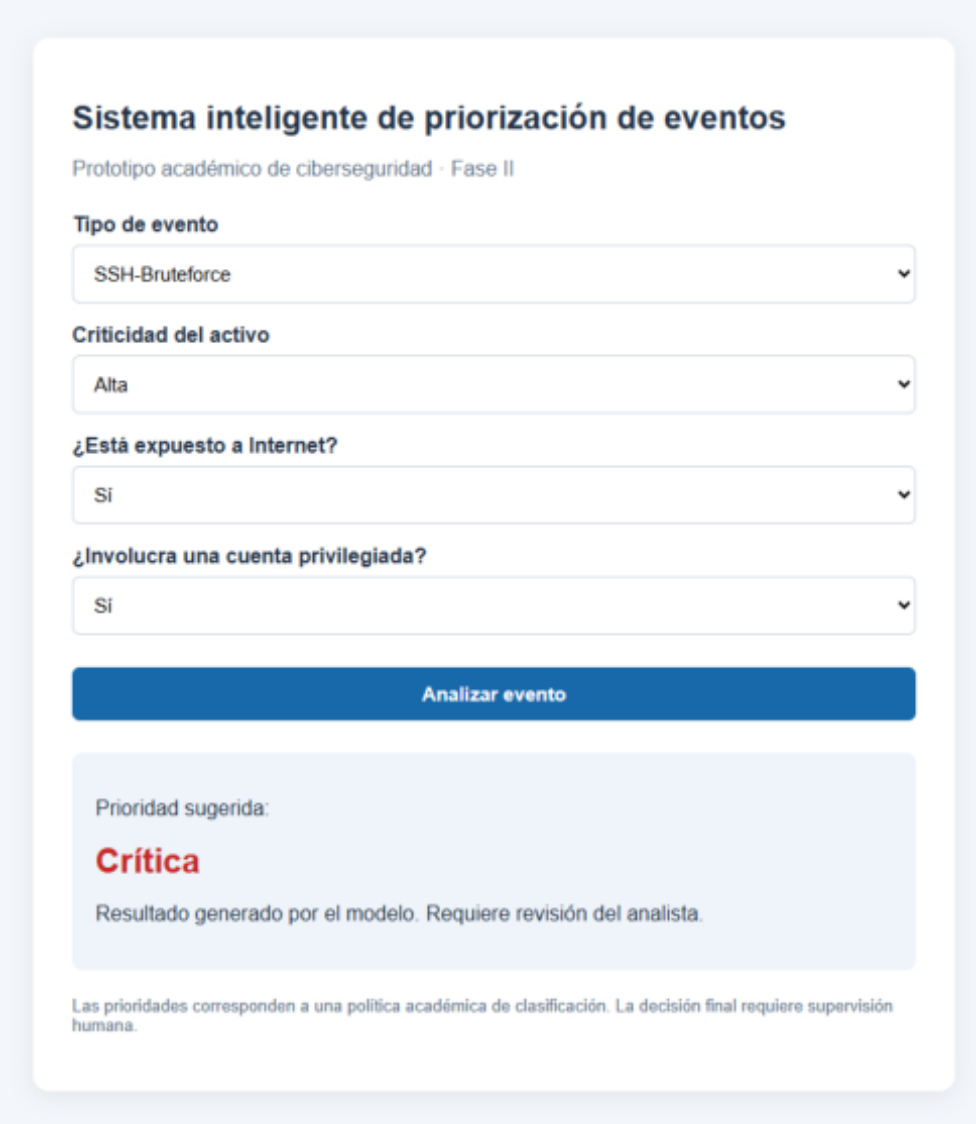


Figura 5. Interfaz Docker: evento SSH-Bruteforce con prioridad Crítica. Fuente: prueba local en localhost:8001.

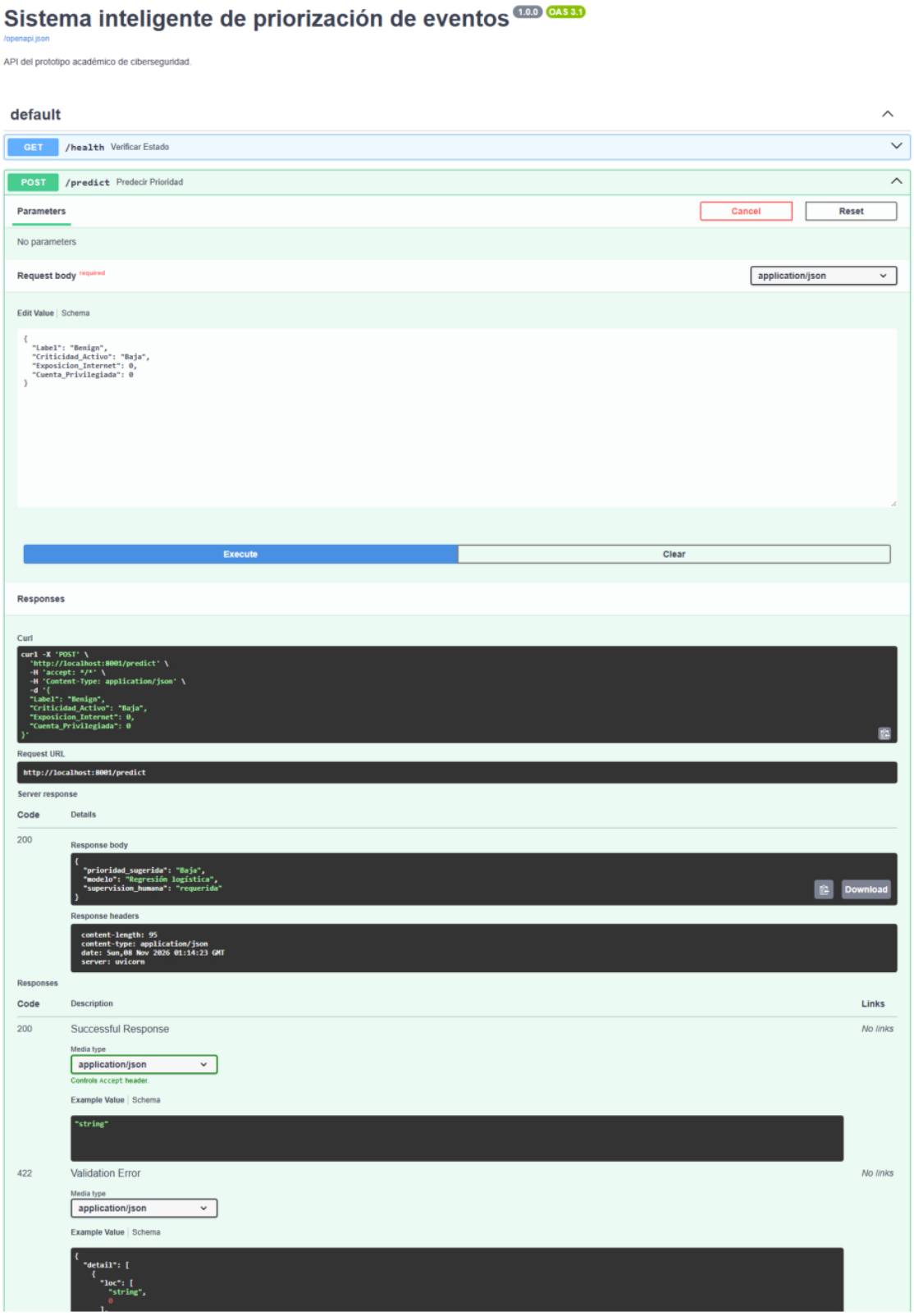


Figura 6. API Docker: escenario de prioridad Baja, respuesta HTTP 200. Fuente: Swagger local.

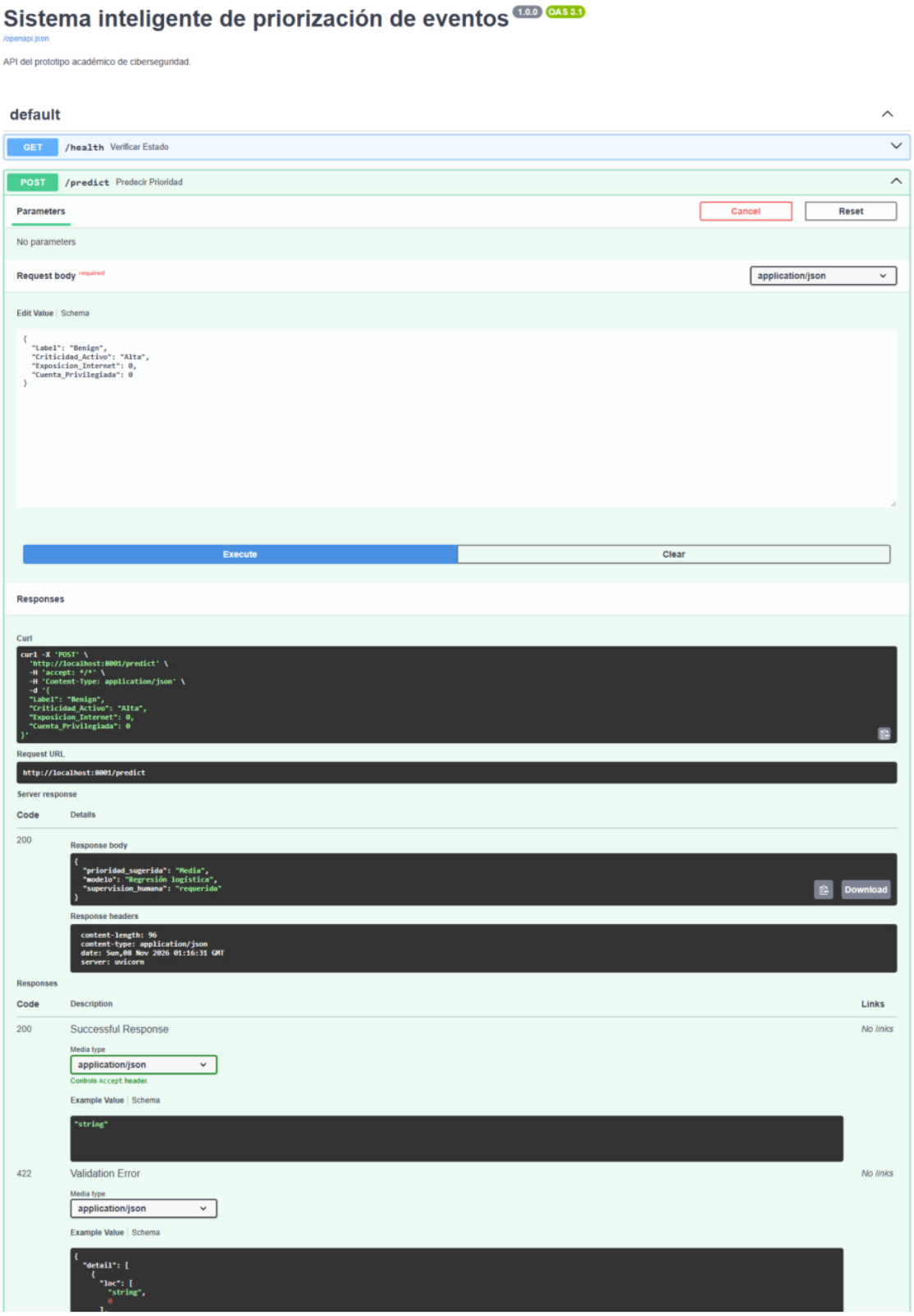


Figura 7. API Docker: escenario de prioridad Media, respuesta HTTP 200. Fuente: Swagger local.

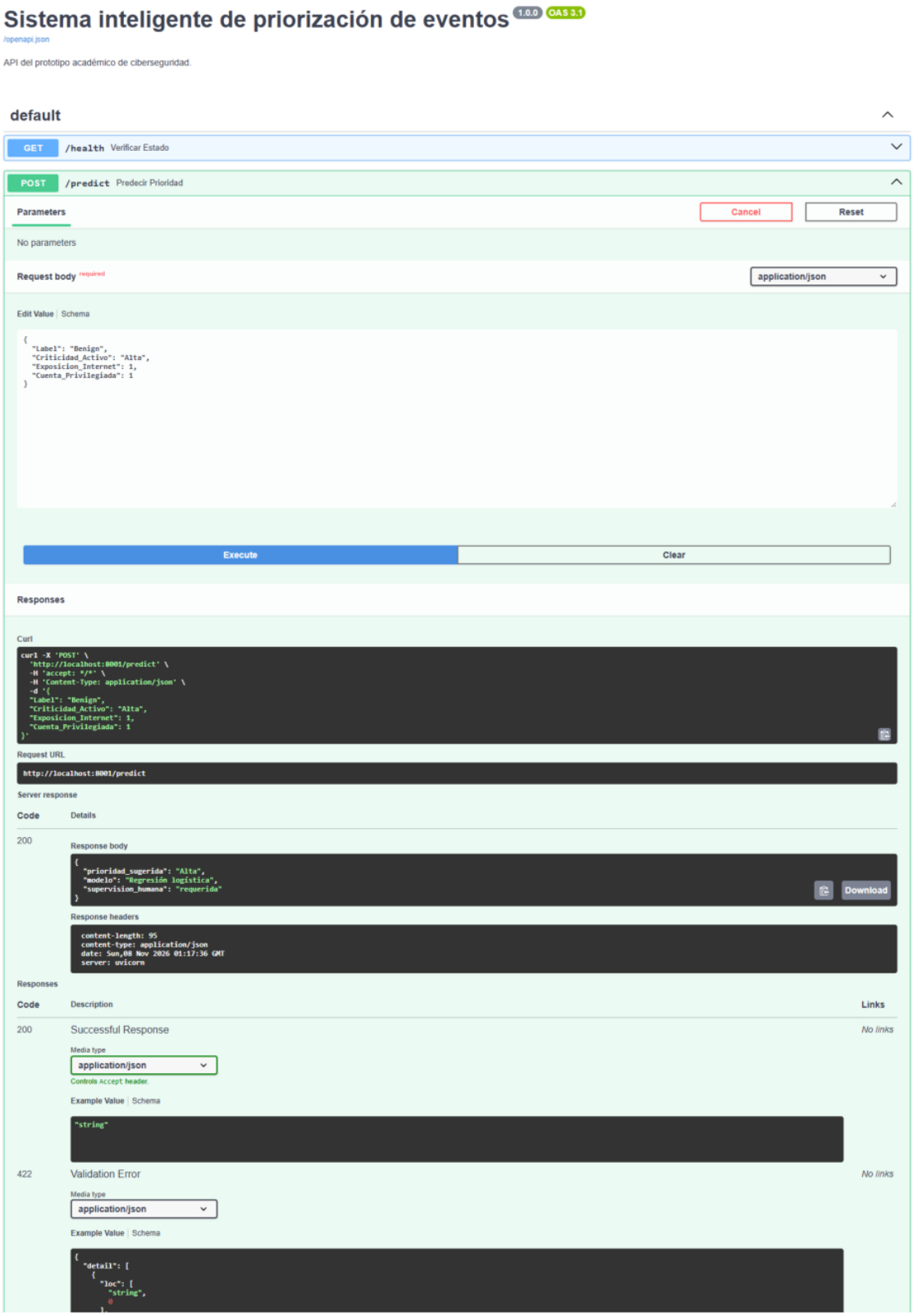


Figura 8. API Docker: escenario de prioridad Alta, respuesta HTTP 200. Fuente: Swagger local.

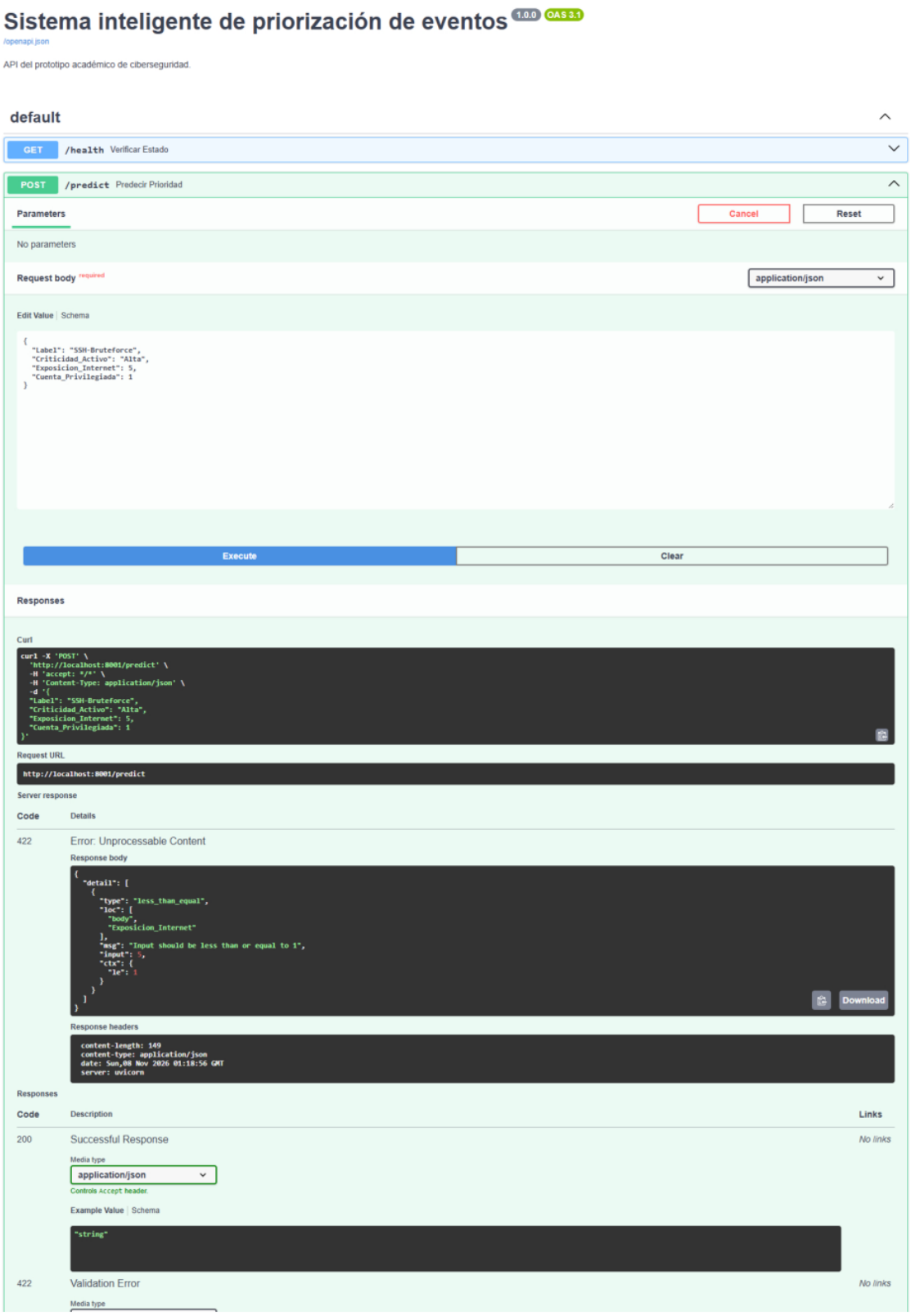


Figura 9. API Docker: rechazo de exposición fuera de rango, HTTP 422. Fuente: Swagger local.

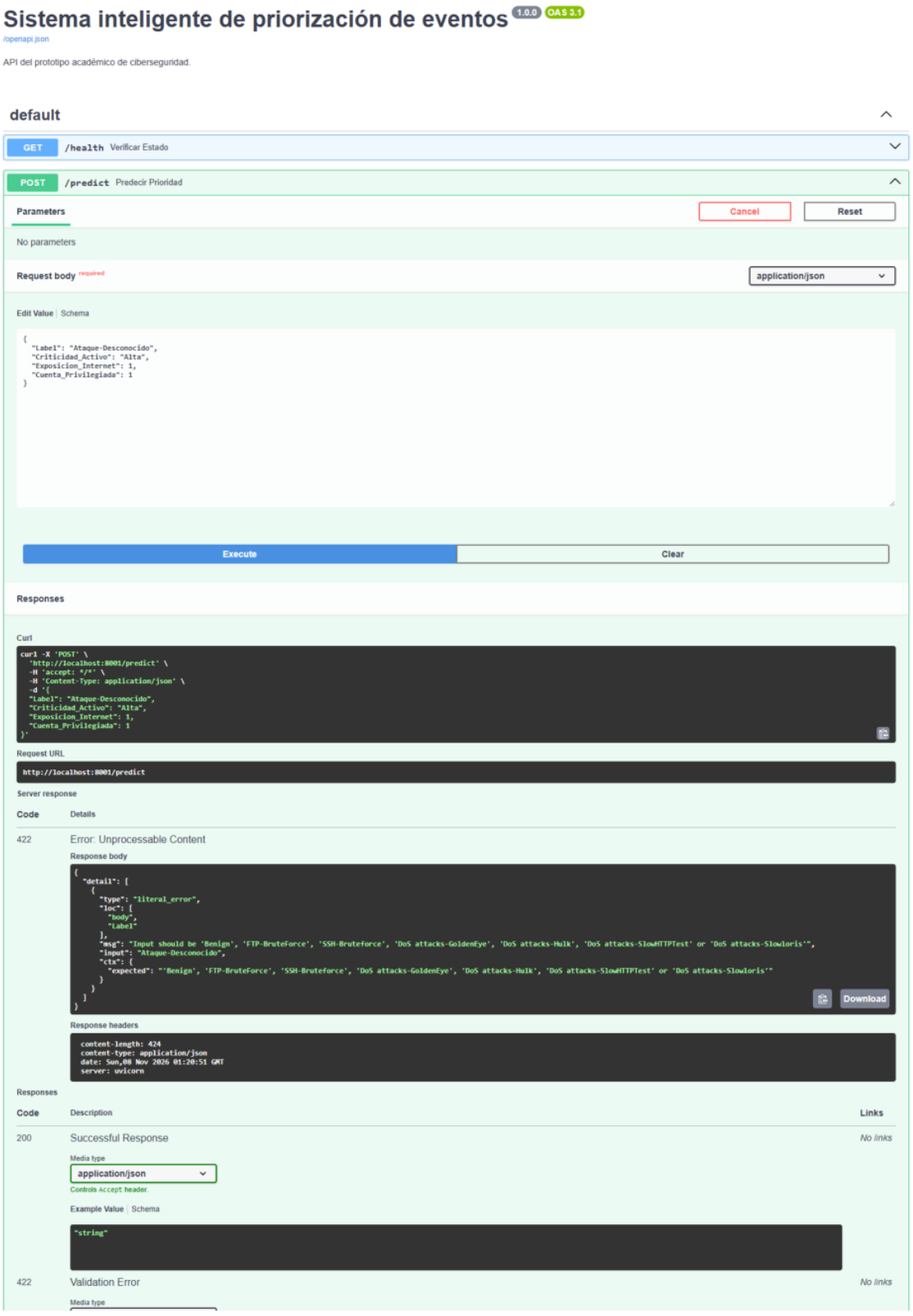
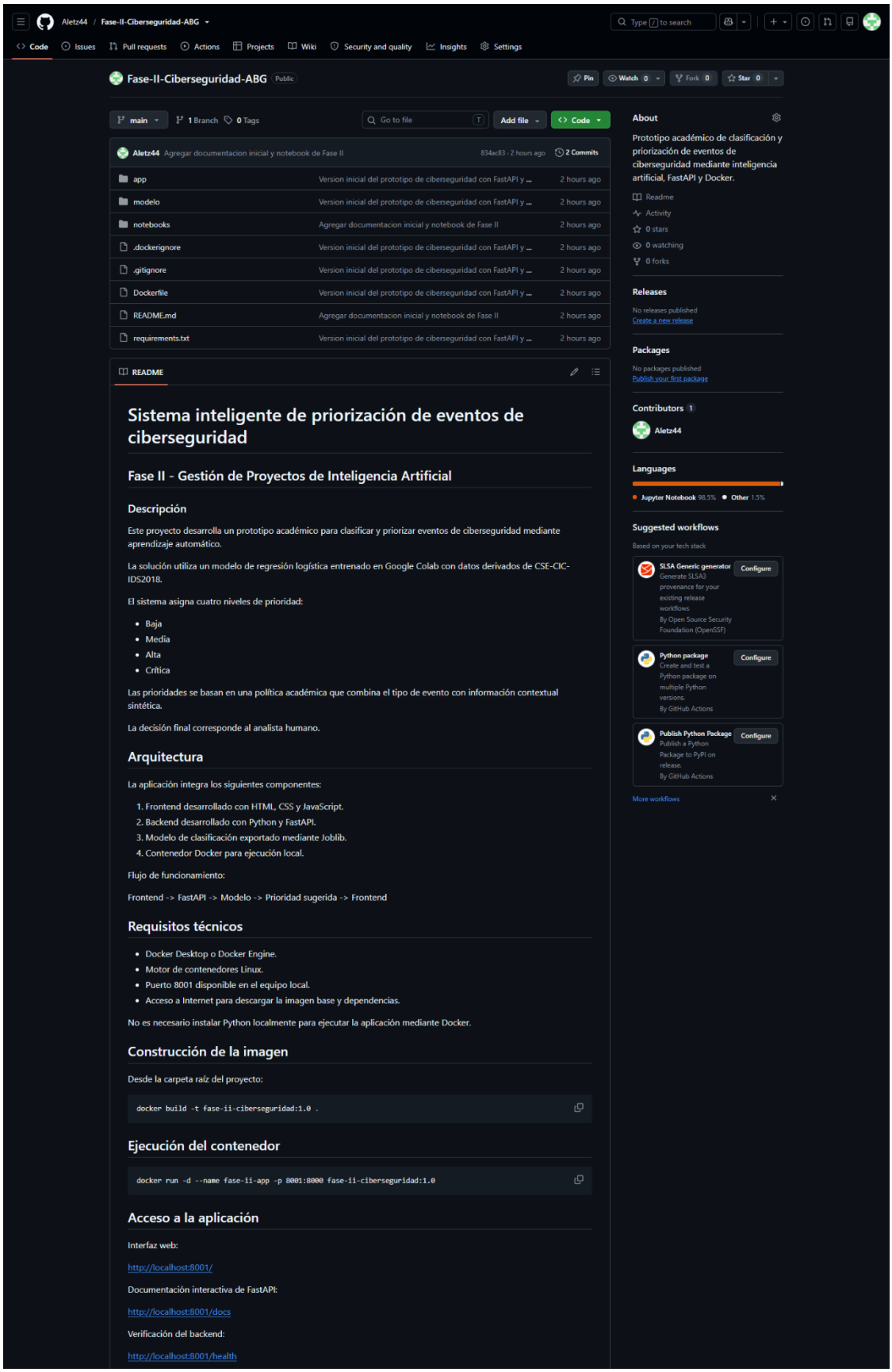


Figura 10. API Docker: rechazo de categoría desconocida, HTTP 422. Fuente: Swagger local.



Aletz44 / Fase-II-Ciberseguridad-ABG

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Fase-II-Ciberseguridad-ABG Public

Pin Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags Go to file Add file Code

Aletz44 Agregar documentacion inicial y notebook de Fase II 834ac83 · 2 hours ago 2 Commits

app	Version inicial del prototipo de ciberseguridad con FastAPI y ...	2 hours ago
modelo	Version inicial del prototipo de ciberseguridad con FastAPI y ...	2 hours ago
notebooks	Agregar documentacion inicial y notebook de Fase II	2 hours ago
.dockerignore	Version inicial del prototipo de ciberseguridad con FastAPI y ...	2 hours ago
.gitignore	Version inicial del prototipo de ciberseguridad con FastAPI y ...	2 hours ago
Dockerfile	Version inicial del prototipo de ciberseguridad con FastAPI y ...	2 hours ago
README.md	Agregar documentacion inicial y notebook de Fase II	2 hours ago
requirements.txt	Version inicial del prototipo de ciberseguridad con FastAPI y ...	2 hours ago

README

Sistema inteligente de priorización de eventos de ciberseguridad

Fase II - Gestión de Proyectos de Inteligencia Artificial

Descripción

Este proyecto desarrolla un prototipo académico para clasificar y priorizar eventos de ciberseguridad mediante aprendizaje automático.

La solución utiliza un modelo de regresión logística entrenado en Google Colab con datos derivados de CSE-CIC-IDS2018.

El sistema asigna cuatro niveles de prioridad:

- Baja
- Media
- Alta
- Crítica

Las prioridades se basan en una política académica que combina el tipo de evento con información contextual sintética.

La decisión final corresponde al analista humano.

Arquitectura

La aplicación integra los siguientes componentes:

1. Frontend desarrollado con HTML, CSS y JavaScript.
2. Backend desarrollado con Python y FastAPI.
3. Modelo de clasificación exportado mediante Joblib.
4. Contenedor Docker para ejecución local.

Flujo de funcionamiento:

Frontend -> FastAPI -> Modelo -> Prioridad sugerida -> Frontend

Requisitos técnicos

- Docker Desktop o Docker Engine.
- Motor de contenedores Linux.
- Puerto 8001 disponible en el equipo local.
- Acceso a Internet para descargar la imagen base y dependencias.

No es necesario instalar Python localmente para ejecutar la aplicación mediante Docker.

Construcción de la imagen

Desde la carpeta raíz del proyecto:

```
docker build -t fase-ii-ciberseguridad:1.0 .
```

Ejecución del contenedor

```
docker run -d --name fase-ii-app -p 8001:8000 fase-ii-ciberseguridad:1.0
```

Acceso a la aplicación

Interfaz web:

http://localhost:8001/

Documentación interactiva de FastAPI:

http://localhost:8001/docs

Verificación del backend:

http://localhost:8001/health

About

Prototipo académico de clasificación y priorización de eventos de ciberseguridad mediante inteligencia artificial, FastAPI y Docker.

Readme
Activity
0 stars
0 watching
0 forks

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Contributors 1

Aletz44

Languages

Jupyter Notebook 98.5% Other 1.5%

Suggested workflows

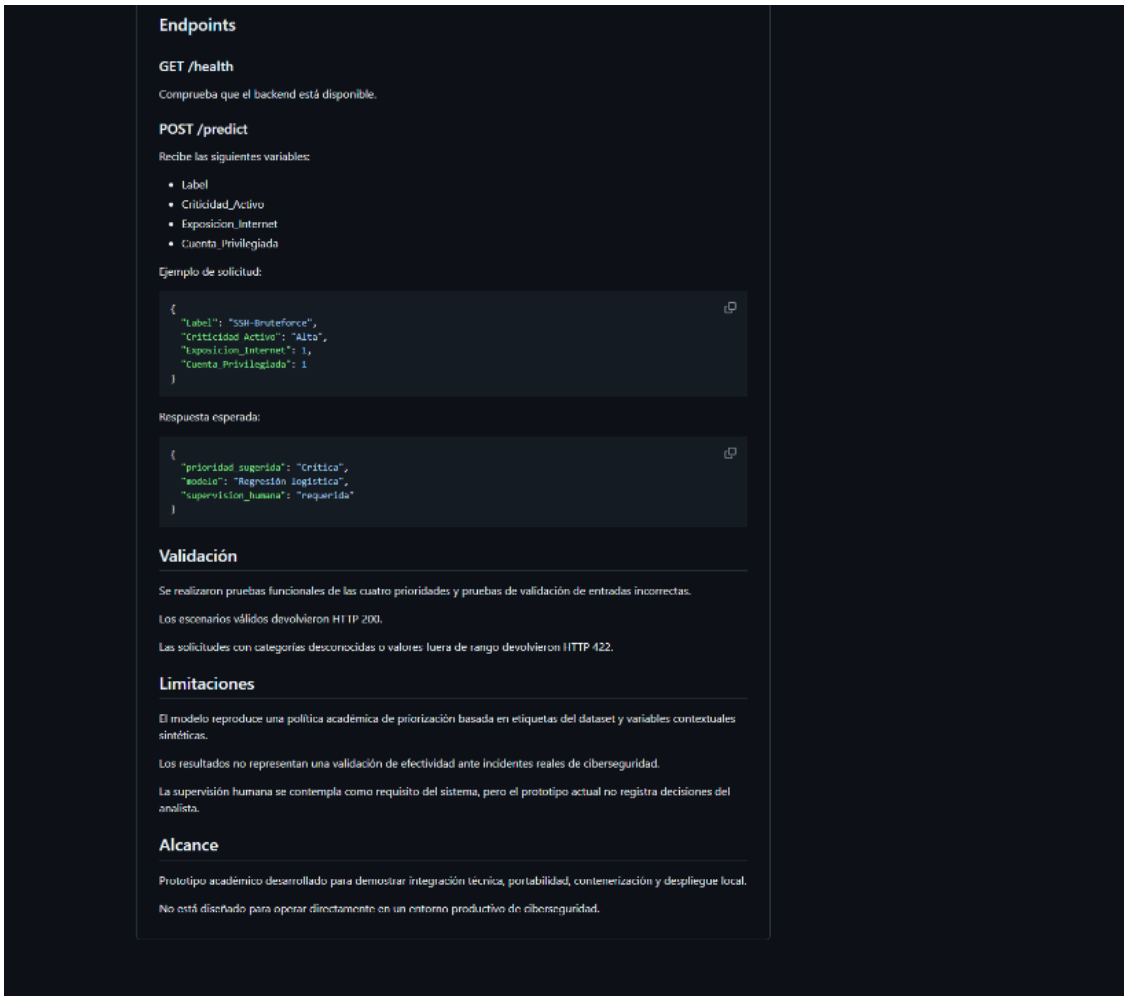
Based on your tech stack

SLSA Generic generator — Configure
Generate SLSA3 provenance for your existing release workflows
By Open Source Security Foundation (OpenSSF)

Python package — Configure
Create and test a Python package on multiple Python versions.
By GitHub Actions

Publish Python Package — Configure
Publish a Python Package to PyPI on release.
By GitHub Actions

More workflows



Endpoints

GET /health

Comprueba que el backend está disponible.

POST /predict

Recibe las siguientes variables:

- Label
- Criticidad_Activo
- Exposicion_Internet
- Cuenta_Privilegiada

Ejemplo de solicitud:

```
{
  "Label": "SSH-Bruteforce",
  "Criticidad_Activo": "Alta",
  "Exposicion_Internet": 1,
  "Cuenta_Privilegiada": 1
}
```

Respuesta esperada:

```
{
  "prioridad_sugerida": "Critica",
  "modelo": "Regresión logistica",
  "supervision_humana": "requerida"
}
```

Validación

Se realizaron pruebas funcionales de las cuatro prioridades y pruebas de validación de entradas incorrectas.

Los escenarios válidos devolvieron HTTP 200.

Las solicitudes con categorías desconocidas o valores fuera de rango devolvieron HTTP 422.

Limitaciones

El modelo reproduce una política académica de priorización basada en etiquetas del dataset y variables contextuales sintéticas.

Los resultados no representan una validación de efectividad ante incidentes reales de ciberseguridad.

La supervisión humana se contempla como requisito del sistema, pero el prototipo actual no registra decisiones del analista.

Alcance

Prototipo académico desarrollado para demostrar integración técnica, portabilidad, contenerización y despliegue local.

No está diseñado para operar directamente en un entorno productivo de ciberseguridad.

Figura 11. Repositorio GitHub público con código y configuraciones. Fuente: captura del repositorio.

Referencias Bibliográficas

Canadian Institute for Cybersecurity. (s. f.). CSE-CIC-IDS2018 on AWS. https://www.unb.ca/cic/datasets/ids-2018.html

Docker Inc. (s. f.). Docker documentation. https://docs.docker.com/

FastAPI. (s. f.). FastAPI documentation. https://fastapi.tiangolo.com/

scikit-learn developers. (s. f.). Model evaluation: Quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html